HemaFlow

FLOW, TRACK, DELIVER

# Implementation Documentation

## HemaFlow

### Group members and subsystems

| Member | Subsystem |
|---|---|
| Amity Brown | Admin & Patient |
| Rohit Aroonslam | Doctor |
| Marcellos von Buchenroder | Lab Manager |
| Bantu-Bethu Beya | Lab Technician |

# Implementation

This document describes how the HemaFlow system was implemented. HemaFlow is a web application for a haematology laboratory. Patients register, keep a medical profile, decide which doctors may see their results, and view, track and download their results as PDF reports. Doctors request tests, Lab Technicians receive samples, capture and verify results, Lab Managers manage the test catalogue, stock, suppliers and staff, and Administrators maintain the master medical data and the user accounts. All five portals read and write one shared SQL Server database, so a patient is the same person on every screen.

The document covers the tools that were chosen and why (1.1), the final relational database model (1.2), extracts of complex code from every subsystem, each with a detailed explanation (1.3), all code, data and material that came from other authors (1.4), the AI tools that were used (1.5) and the problems that we met while building the system and how we solved them (1.6).

The work was divided between the four group members as shown on the cover page: Amity Brown built the Admin and Patient subsystems, Rohit Aroonslam the Doctor subsystem, Bantu-Bethu Beya the Lab Technician subsystem and Marcellos von Buchenroder the Lab Manager subsystem.

## 1.1 Choice of Tools

### 1.1.1 Visual Studio 2022

Visual Studio 2022 is the IDE used to write, build and debug the whole application. It has first-class support for ASP.NET Core, Razor views, NuGet packages and the Git integration the group used to share code. The Razor editor catches mistakes in views while typing, and the debugger made it possible to step through problems such as the consent and approval logic. The release build for the campus server was published with the .NET command line from the same solution.

### 1.1.2 SQL Server 2022 and SQL Server Management Studio

All data is stored in SQL Server 2022. This version was chosen because the campus server (soit-sql.mandela.ac.za) runs SQL Server 2022, and a database built on a newer version cannot be attached to an older one. SQL Server Management Studio (SSMS) was used to run the hand-written SQL scripts that create the tables and seed the data, to inspect rows while testing (for example the IsArchived flag on users), and to check that the submission database could be attached under the name GRP-04-13.

### 1.1.3 ASP.NET Core MVC (.NET 8), C# and Razor

The application is an ASP.NET Core MVC web application on .NET 8 written in C#. MVC separates the screens (Razor views), the logic (controllers) and the data (models), which let each member work on their own controllers and views with little overlap. Role-based authorization with [Authorize(Roles = ...)] on every controller was a strong reason for the choice: Administrator, Patient, Doctor, Lab Manager and Lab Technician each only reach their own pages. Razor tag helpers produce correct links even though the site runs in an IIS virtual application.

### 1.1.4 Entity Framework Core 8

Entity Framework Core 8 with the SQL Server provider is the object-relational mapper. The database was built first from SQL scripts and the entity classes were scaffolded from it (database-first), then edited by hand where needed. One shared ApplicationDbContext is used by every subsystem. LINQ queries replace hand-written SQL in the controllers, which removes the risk of SQL injection and keeps the code readable. Migrations were deliberately not used, because the group agreed that every schema change is a numbered SQL script that can be rerun on any machine.

### 1.1.5 ASP.NET Core cookie authentication and PasswordHasher

Users sign in with an e-mail and password. The password is checked against a salted hash made by the ASP.NET Core Identity PasswordHasher, so plain passwords are never stored. After a successful sign-in the server issues an authentication cookie that carries the user id, name, e-mail and role. A second short-lived cookie scheme holds the user between the password step and the two-factor code step. A small piece of middleware sends the "no-store" cache header on every page for a signed-in user, so the browser Back button cannot show a private page after logging out.

### 1.1.6 Authenticator-app two-factor authentication (own TOTP code)

Administrators, patients and doctors can turn on two-factor authentication with an authenticator app. The time-based one-time password (TOTP) algorithm from RFC 6238 was written by hand using the HMAC classes in .NET, so no extra package was needed. A user scans a QR code, confirms with a code, receives backup codes (only their hashes are stored), and from then on is asked for a code at sign-in.

### 1.1.7 QuestPDF

Every report (patient result reports, the admin master-data and user exports, and the doctor and laboratory reports) is produced as a designed PDF with QuestPDF, so the reports follow the ONP400 report guidelines: a title, the generated date and filters, a header and a footer with page numbers, repeated table headings and a "No records found" page when a range is empty. QuestPDF describes pages in C# code, which let all roles share one PdfService and look consistent.

### 1.1.8 Brevo SMTP e-mail

The system sends e-mail for forgotten-password links, temporary passwords for accounts that staff create, and notifications to a doctor when a patient grants access. Brevo provides an SMTP relay that works from the campus server. The settings are read from the Smtp section of the configuration, and the sender is a thin wrapper around System.Net.Mail so no extra package was needed.

### 1.1.9 Bootstrap 5 and Bootstrap Icons

Bootstrap provides the responsive grid, forms, modals, dropdowns and buttons, and Bootstrap Icons provides the icons in the menus, buttons and status labels. Together they gave a consistent look on every portal with little custom CSS, and the pages stay usable on a phone-sized screen. Both libraries are stored in the project (wwwroot/lib) instead of loaded from the internet, so the system still looks right on a network with limited access.

### 1.1.10 Chart.js

The Track Results page on the patient portal draws each test result over time as a line chart, with the normal minimum and maximum and the patient's own median as reference lines. Chart.js was chosen because it draws responsive charts on a canvas from a small amount of JavaScript and supports tooltips and legends. A local copy is used for the same reason as Bootstrap.

### 1.1.11 QRCode.js

The two-factor setup page shows the otpauth link as a QR code that an authenticator app can scan. QRCode.js draws that code in the browser, so the secret key never has to be sent to a third-party service to make an image.

### 1.1.12 Inter and Sora fonts

Inter is the text font on every page and Sora is used for the HemaFlow name on the public pages. Both are served from the project folder as web font files, which avoids a request to an external font service and gives the same look on every machine.

### 1.1.13 GeoNames postal code data

South African addresses are stored as Province, Town and Suburb (with postal code) instead of free text. GeoNames postal code data was used to build the suburb list behind the three linked drop-down boxes on the registration and profile pages. The data is used under the CC BY 4.0 licence and credited in section 1.4.

### 1.1.14 Internet Information Services (IIS) and the campus server

The application was published in Release mode with dotnet publish and copied to the group's IIS site on the campus server. IIS hosts the application under a virtual path, which is why the code reads ASPNETCORE_APPL_PATH. The campus database is attached under the group name GRP-04-13.

### 1.1.15 Git and GitHub

Git was used for version control so that four people could work on one solution at the same time, and GitHub was used to host the shared repository. Every member committed their own subsystem to the repository, which gave us a history of all changes, made it possible to go back after a mistake, and let the group merge the Admin, Patient, Doctor, Lab Technician and Lab Manager work into one project that builds and runs as a whole.

### 1.1.16 Microsoft Azure

Microsoft Azure was used by the group together with GitHub for version control and for organising our work, so that all four members always worked on the same, up-to-date copy of the project and could see each other's changes.

## 1.2 Relational Database Model Diagram

Figure 1.1 represents the relational database model diagram of the HemaFlow system. It shows all 50 tables of the shared database, drawn in crow's foot notation, with the primary key (PK) and foreign key (FK) columns marked and every relationship labelled. The full-size diagram is also supplied as a separate file, **HemaFlow_ERD.pdf**, attached separately with this document, so it can be zoomed in to read every column. Figures 1.2 and 1.3 on the next pages show the left and the right half of the same diagram at a larger size.

The Users table is the centre of the model: everyone who can sign in has one row in User with a role, and the details for each role are in its own profile table (AdminProfile, PatientProfile, DoctorProfile, LabManagerProfile and TechnicianProfile). Addresses are normalised through Suburb, Town and Province. Patients' conditions, allergies and medications link to the master lists, which are grouped by Category, and anything a patient types in that is not on a list waits in PendingApproval for an administrator. Consent between patients and doctors is stored in PatientDoctorConsent and ConsentedTestRequest. A test request has one or more items, each item has samples and one result, and every result can have verification records. Stock, suppliers and orders form their own area, and staff and patient actions are written to the two activity log tables.

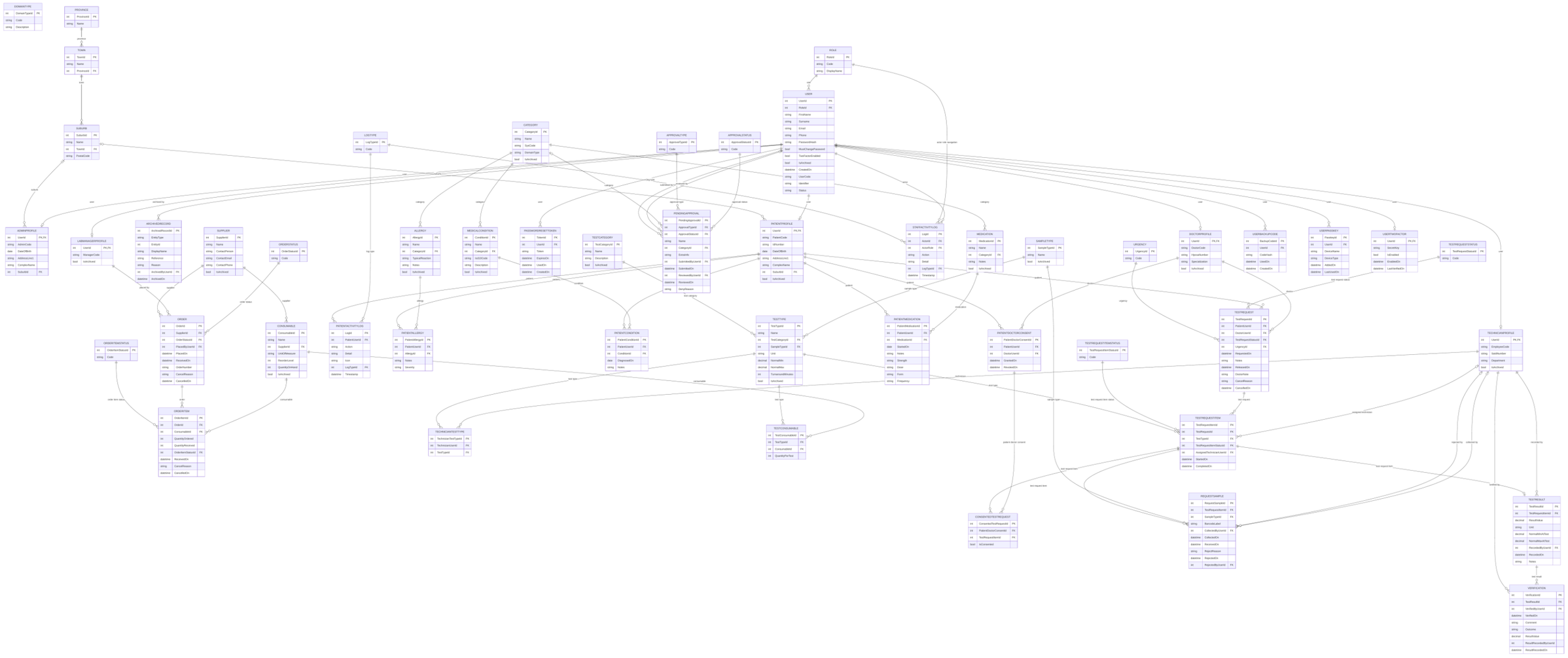



*Figure 1.1 Relational Database Model Diagram (full diagram)*

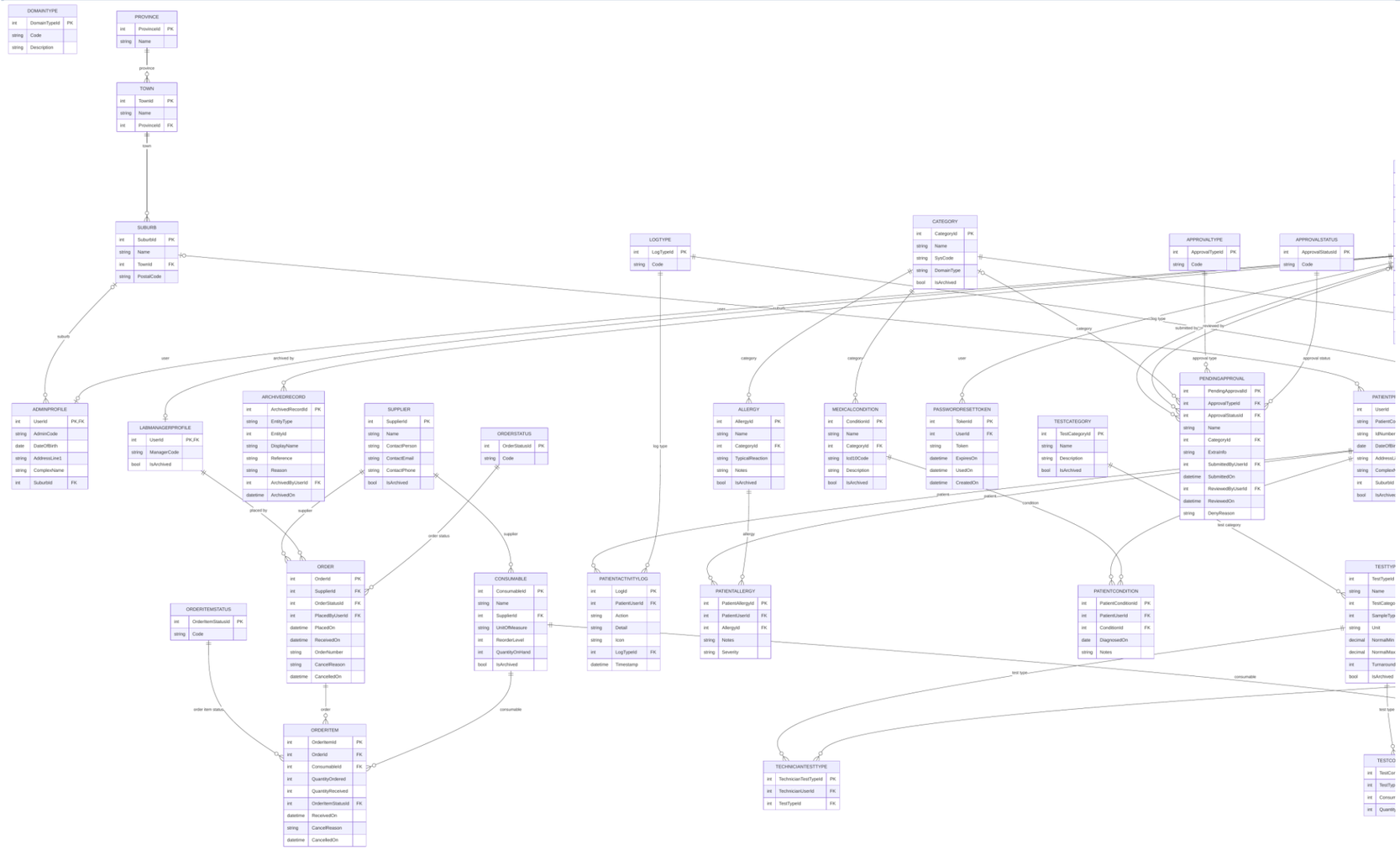



*Figure 1.2 Relational Database Model Diagram (left half of the diagram)*

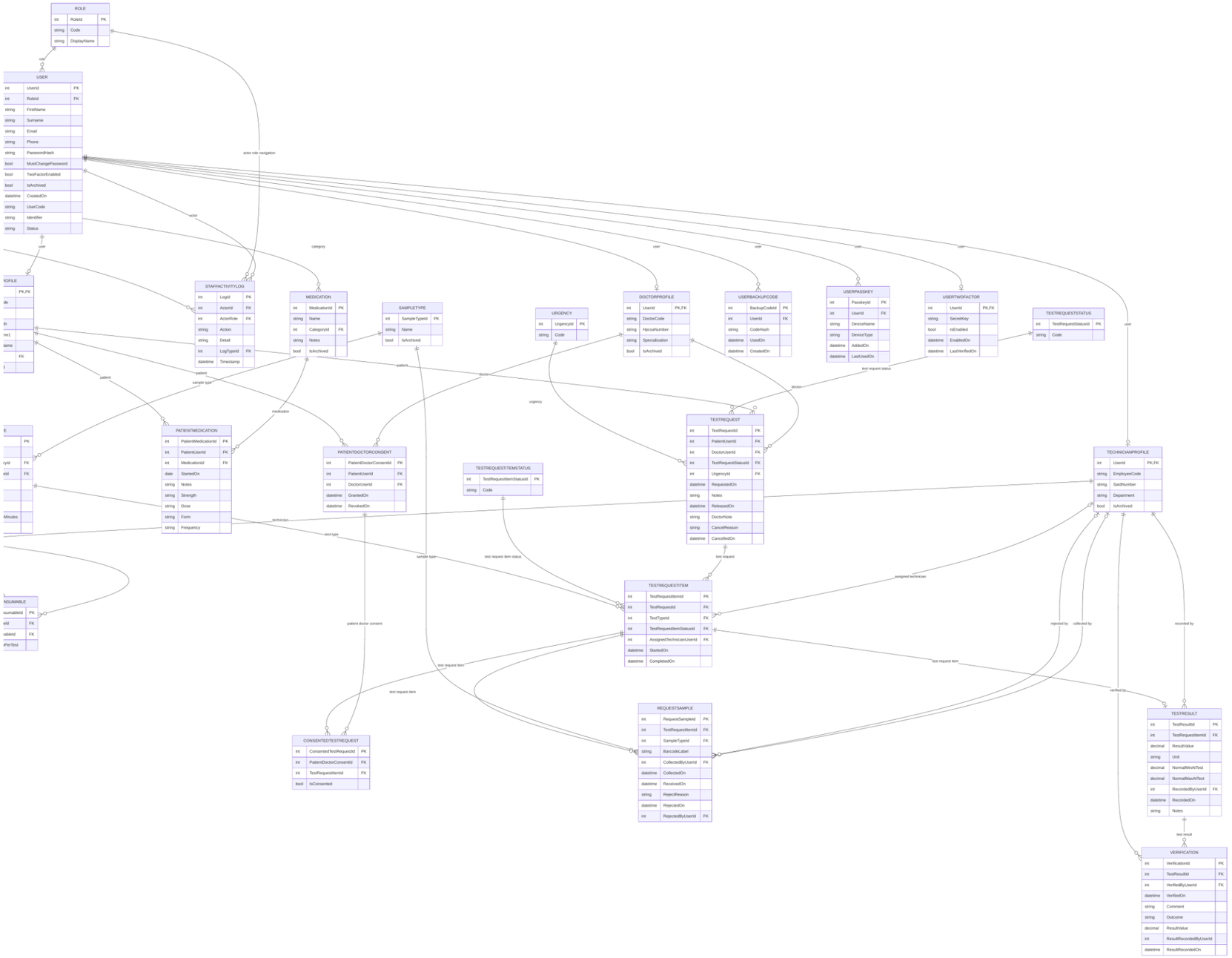



*Figure 1.3 Relational Database Model Diagram (right half of the diagram)*

## 1.3 Extracts of complex code

The extracts below are from each of the five subsystems. Every extract is marked with the subsystem it belongs to, followed by a detailed explanation of what the code does. All of the code in this section was written by us.

### 1.3.1 Approving a patient request and attaching it to that patient (Admin)

```
// The admin may correct the type (e.g. a patient filed a medication under "allergy") and the name.
var newTypeId = entityType switch { "Condition" => (byte)1, "Allergy" => (byte)2, "Medication" => (byte)3, _
=> item.ApprovalTypeId };
item.ApprovalTypeId = newTypeId;
var cleanName = (name ?? "").Trim();
if (cleanName.Length > 200) { TempData["Error"] = "Name is too long (200 characters maximum)."; return
RedirectToAction("PendingApprovals"); }
if (cleanName.Length > 0) item.Name = cleanName;

entityType = ApprovalTypeToEntityType(item.ApprovalTypeId);
// Only categories of the matching kind are valid (a category name can exist under several kinds).
var domain = entityType == "Condition" ? "Medical" : entityType;
var catsOfDomain = GetAllCats().Where(c => c.DomainType == domain && !c.IsArchived).ToList();
if (!catsOfDomain.Any(c => c.Name == category)) { TempData["Error"] = $"'{category}' is not a
{entityType.ToLower()} category."; return RedirectToAction("PendingApprovals"); }
switch (entityType)
{
    case "Condition":
        var condCatId = catsOfDomain.FirstOrDefault(c => c.Name == category)?.Id ?? 0;
        var newCondition = _context.MedicalConditions.FirstOrDefault(x => x.Name == item.Name);

if (newCondition != null) newCondition.IsArchived = false; // already in the catalogue: reuse it
else
{
    if (condCatId == 0) { TempData["Error"] = $"Category '{category}' was not found."; return
RedirectToAction("PendingApprovals"); }
    newCondition = new GenMedicalCondition { Name = item.Name, CategoryId = condCatId, Icd10Code = "",
Description = "Added via patient-submitted request.", IsArchived = false };
    _context.MedicalConditions.Add(newCondition);
}
_context.SaveChanges(); // need the generated ConditionId before attaching it to the patient

// Attach straight to the submitting patient's own profile,
// not just the master list - item.SubmittedByUserId is the
// real Users.UserId captured at submission time.
if (!_context.PatientConditions.Any(pc => pc.PatientUserId == item.SubmittedByUserId && pc.ConditionId ==
newCondition.ConditionId))
{
    _context.PatientConditions.Add(new GenPatientCondition
    {
        PatientUserId = item.SubmittedByUserId,
        ConditionId = newCondition.ConditionId,
        DiagnosedOn = DateOnly.FromDateTime(DateTime.Today)
    });
    _context.SaveChanges();
}

// ... the Allergy and Medication cases repeat the same pattern ...

item.ApprovalStatusId = 2; // Approved
item.ReviewedByUserId = CurrentUserId;
item.ReviewedOn = DateTime.UtcNow;
_context.SaveChanges();

LogAction(CurrentUserEmail, CurrentUserRoleCode, $"{entityType} approved", $"Patient-submitted
{entityType.ToLower()} '{item.Name}' approved and added (category: {category})", "success");
TempData["Success"] = $"'{item.Name}' approved and added to the master list.";
```

*Figure 1.4 ApprovePending: approving a patient-submitted item*

When a patient cannot find a condition, allergy or medication in the lists they choose "Other" and type it in. The request waits in the PendingApproval table until an administrator approves it. This method runs when the administrator presses Approve and does three jobs in one go: it adds the item to the master list, it adds it to that patient's own medical profile, and it records the decision.

The method first lets the administrator correct the request. The switch expression turns the chosen type (Condition, Allergy or Medication) into the numeric approval type stored in the table, so a patient who filed a medication under "allergy" can be fixed, and the new name is trimmed and limited to 200 characters. Next the chosen category is checked against the correct kind of category. A category name can exist under more than one kind, so the list of categories is filtered by the domain ("Condition" is stored as "Medical") and archived categories are left out; if the category does not belong, an error message is shown and nothing is saved. This stops a medication from being filed under an allergy category.

In the Condition case the code first looks for a condition with the same name. If it already exists (for example it was archived earlier) it is reused and un-archived, which prevents duplicates in the master list; otherwise a new MedicalCondition row is created with the chosen category. SaveChanges is called straight away because the database generates the new ConditionId and that value is needed in the next step. The condition is then attached to the patient with the id in item.SubmittedByUserId. That value is the real Users.UserId saved when the patient submitted the request, so no name lookup is needed, which matters because two patients can share a name. The Any(...) check makes the step safe to repeat, and DiagnosedOn needs DateOnly.FromDateTime because the column is a SQL DATE.

Finally the request is marked as approved (status 2) together with the reviewing administrator and the UTC time of the decision, the action is written to the system activity log under the administrator's own identity, and a success message is shown. The Allergy and Medication cases follow the same pattern; for a medication the text the patient typed for strength and frequency is kept in the Notes column.

### 1.3.2 Deciding who may manage which account (Admin)

```
private List<UserRow> GetManagedUsers()
{
    var superCodes = _context.Users.Where(u => u.Role.Code == "SuperAdmin").Select(u => u.UserCode).ToList();
    var rows = GetUsers().Where(u => u.Role != "Patient" && u.Role != "Doctor" &&
!superCodes.Contains(u.UserId)).ToList();
    foreach (var r in rows)
        r.CanManage = !r.IsArchived && CanManageRole(r.RoleCode);
    return rows;
}


// Only a Super Admin may manage Admin accounts. Admins manage Lab Technicians and Lab Managers.
private bool CanManageRole(string roleCode) =>
    User.IsInRole("SuperAdmin")
        ? roleCode != "SuperAdmin"
        : roleCode == "Technician" || roleCode == "LabManager";


// Returns an error message when the signed-in admin may not change this account, otherwise null.
private string? ManageDeniedMessage(GenUser target)
{
    if (target.IsArchived) return "Archived accounts cannot be changed here.";
    if (!CanManageRole(target.Role.Code))
        return target.Role.Code == "Admin"
            ? "Only a Super Admin can manage Admin accounts."
            : "You cannot manage this account.";
    return null;
}
```

*Figure 1.5 GetManagedUsers, CanManageRole and ManageDeniedMessage*

User Management must follow a business rule: only a Super Admin may manage Administrator accounts, while a normal Administrator may only manage Lab Technicians and Lab Managers. Archived accounts cannot be changed by anyone, and Super Admin accounts can never be changed from the screen.

CanManageRole holds the rule in one place. If the signed-in user is a Super Admin the answer is true for every role except SuperAdmin; for a normal Administrator it is true only for Technician and LabManager. GetManagedUsers builds the list shown on the page. It first collects the codes of the Super Admin accounts and removes them from the list together with patients and doctors, so nobody can even see them. For every remaining row it sets CanManage to true only when the account is not archived and CanManageRole allows the role. The view shows the Manage menu only when CanManage is true; otherwise it shows "Archived" or "Super Admin only".

Hiding a button is not security, because a person can post a form by hand. For that reason every action that changes an account (edit, reset password, deactivate, activate) loads the target account from the database and calls ManageDeniedMessage. That method returns a message when the change is not allowed (an archived account, or an Admin account for a normal Administrator) and null when it is allowed, so the same rule is enforced on the server whatever the browser sends.

### 1.3.3 Storing time in UTC and showing it in South African time (Admin & Patient)

```
namespace Project.Services
{
    // Everything is stored in the database as UTC; people in South Africa see it as SAST (UTC+2, no daylight
saving).
    public static class SastTime
    {
        public static readonly TimeSpan Offset = TimeSpan.FromHours(2);


        public     static     DateTime     ToSast(DateTime     stored)     =>     DateTime.SpecifyKind(stored,
DateTimeKind.Unspecified) + Offset;


        // A date the person typed (in SAST) -> the UTC moment it starts at.
        public static DateTime SastDateToUtc(DateTime sastDate) => sastDate.Date - Offset;
    }
```

*Figure 1.6 SastTime*

Every date and time is stored in the database as UTC so that the data does not depend on the time zone of the server. People in South Africa expect to see local time (SAST, UTC+2, with no daylight saving), so every screen converts before it displays a time.

ToSast adds the two-hour offset to a stored value. DateTime.SpecifyKind with Unspecified is used first so that the .NET runtime does not apply a second conversion with the server's own time zone, which would give different answers on different machines. SastDateToUtc does the opposite for filters: when an administrator picks a date in the activity log, that date means the start of that day in South Africa, so two hours are subtracted to get the UTC moment that the database rows can be compared with. Using these two small methods everywhere is what makes the activity logs, archives, the patient activity log and the reports all show the same, correct time.

### 1.3.4 Adding the treating doctors to the consent page automatically (Patient)

```
// Every doctor who has requested tests for this patient shows up on the Consent page.
// Doctors the patient already revoked are left alone, and the 5 doctor limit is respected.
private void SyncTreatingDoctors()
{
    if (CurrentUserId == null) return;
    var uid = CurrentUserId.Value;
    var requested = _context.TestRequests.AsNoTracking()
        .Where(r => r.PatientUserId == uid)
        .Select(r => new { r.DoctorUserId, r.TestRequestId }).ToList();
    var known = _context.PatientDoctorConsents.Where(c => c.PatientUserId == uid).Select(c => c.DoctorUserId).ToList();
    int active = _context.PatientDoctorConsents.Count(c => c.PatientUserId == uid && c.RevokedOn == null);
    bool added = false;
    foreach (var docId in requested.Select(r => r.DoctorUserId).Distinct().Except(known))
    {
        if (active >= 5) break;
        var dn = _context.Users.Where(u => u.UserId == docId).Select(u => u.FirstName + " " + u.Surname).FirstOrDefault();
        _context.PatientActivityLogs.Add(new Project.Models.Entities.PatientActivityLog
        {
            PatientUserId = uid, Action = "Doctor added",
            Detail = $"Dr. {dn} was added to your doctors because they requested tests for you.", Icon = "bi-person-check-fill", LogTypeId = 1
        });
        var consent = new Project.Models.Entities.PatientDoctorConsent { PatientUserId = uid, DoctorUserId = docId, GrantedOn = DateTime.UtcNow };
        _context.PatientDoctorConsents.Add(consent);
        // Access only to the tests this doctor ordered, not to everything.
        var ids = requested.Where(r => r.DoctorUserId == docId).Select(r => r.TestRequestId).ToList();
        var items = _context.TestRequestItems.Where(i => ids.Contains(i.TestRequestId)).Select(i => i.TestRequestItemId).ToList();
        foreach (var itemId in items)
            _context.ConsentedTestRequests.Add(new Project.Models.Entities.ConsentedTestRequest { PatientDoctorConsent = consent, TestRequestItemId = itemId, IsConsented = true });
        active++; added = true;
    }
    if (added) { _context.SaveChanges(); _consentCache = null; }
```

*Figure 1.7 SyncTreatingDoctors*

A patient decides which doctors may see their results, and this is stored in PatientDoctorConsent (one row per patient and doctor) and ConsentedTestRequest (one row per test the doctor may see). A doctor who orders tests for a patient must appear on that patient's Consent page, otherwise the patient cannot see who is testing them and cannot revoke the access. This method is called every time the Consent page opens and makes sure that every doctor with a test request is listed.

The method reads the patient's requests (only the doctor and request ids, with AsNoTracking because nothing is changed), then the doctors who already have a consent row. That list includes revoked rows, and this is deliberate: LINQ's Except removes every doctor who already has a row from the list of doctors to add, so a doctor the patient has revoked is never added again behind the patient's back. The variable active counts the doctors who currently have access, and the loop stops at five because the system allows a maximum of five doctors for a patient.

For every new doctor three things are created before one SaveChanges call. A "Doctor added" entry goes into the patient's activity log with the doctor's name. A consent row is added with the grant time in UTC. Then one ConsentedTestRequest row is added for every test item in that doctor's requests. In this model a consent without any ConsentedTestRequest rows means full access (a primary doctor), so adding the rows limits the doctor to only the tests they ordered. After saving, the cached list of consents is cleared so the page that follows shows the new doctors.

### 1.3.5 Time-based one-time passwords for two-factor sign-in (Admin & Patient)

```
// Allows the code from one step before/after the current one, since the
// user's clock and the server's clock are never perfectly in sync.
public static bool ValidateCode(string base32Secret, string? code, int allowedDriftSteps = 1)
{
    if (string.IsNullOrWhiteSpace(code)) return false;
    code = code.Trim();
    if (code.Length != Digits || !code.All(char.IsDigit)) return false;


    var keyBytes = Base32Decode(base32Secret);
    var currentCounter = GetCounter(DateTime.UtcNow);


    for (var drift = -allowedDriftSteps; drift <= allowedDriftSteps; drift++)
    {
        var candidate = ComputeTotp(keyBytes, currentCounter + drift);
        if (candidate == code) return true;
    }
    return false;
}


{
    var unixSeconds = new DateTimeOffset(utcTime).ToUnixTimeSeconds();
    return unixSeconds / PeriodSeconds;
}


private static string ComputeTotp(byte[] key, long counter)
{
    var counterBytes = BitConverter.GetBytes(counter);
    if (BitConverter.IsLittleEndian) Array.Reverse(counterBytes); // RFC requires big-endian


    using var hmac = new HMACSHA1(key);
    var hash = hmac.ComputeHash(counterBytes);


    // Dynamic truncation (RFC 4226 section 5.3)
    var offset = hash[^1] & 0x0F;
    var binaryCode = ((hash[offset] & 0x7F) << 24)
                   | ((hash[offset + 1] & 0xFF) << 16)
                   | ((hash[offset + 2] & 0xFF) << 8)
                   | (hash[offset + 3] & 0xFF);
```

*Figure 1.8 TotpService.ValidateCode and ComputeTotp*

Two-factor authentication asks for a six-digit code from an authenticator app after the password. The app and the server share a secret key and both calculate the same code from the current time, following RFC 6238. TotpService implements the algorithm with only the standard HMAC class, so it is not a copy of a library.

ValidateCode first rejects anything that is not exactly six digits, so a blank or malformed code never reaches the calculation. The secret is stored as Base32 text and decoded to bytes. The time is turned into a counter by dividing the Unix time in seconds by 30, so the counter changes every 30 seconds. Because the clock of the phone and the clock of the server are never exactly equal, the code checks the counters one step before, the current one and one step after (allowedDriftSteps = 1) and accepts the code if any of them matches.

ComputeTotp turns a counter into a code. The counter is written as eight bytes in big-endian order (the bytes are reversed on a little-endian machine, as the RFC requires) and an HMAC-SHA1 is calculated over them with the secret as the key. Dynamic truncation then picks four bytes from the 20-byte result: the last four bits of the last byte give the starting position, and the highest bit of the first picked byte is masked off (& 0x7F) so that the number is positive. The number modulo 1 000 000 gives six digits, padded with leading zeros. Because this is the standard algorithm, the codes match those shown by Google Authenticator and Microsoft

Authenticator. A known limitation is that the same code can be used again inside its 30-second window; this is recorded as a future improvement.

### 1.3.6 Saving a normalised address from the drop-down boxes or typed text (Admin & Patient)

```
public static int? ResolveOrCreate(ApplicationDbContext db, int? suburbId, int? provinceId, string? town,
string? suburb, string? postal, out string? error)
{
    error = null;
    if (suburbId is int sid && sid > 0)
    {
        if (db.Suburbs.Any(s => s.SuburbId == sid)) return sid;
        error = "Please choose your suburb from the list.";
        return null;
    }
    town = (town ?? "").Trim(); suburb = (suburb ?? "").Trim(); postal = (postal ?? "").Trim();
    if (provinceId == null || provinceId <= 0 || town.Length == 0 || suburb.Length == 0)
    { error = "Please choose your province, town and suburb."; return null; }
    if (postal.Length != 4 || !postal.All(char.IsDigit))
    { error = "Postal code must be 4 digits."; return null; }
    if (town.Length > 100 || suburb.Length > 100) { error = "Town or suburb name is too long."; return null; }


    var pid = (byte)provinceId.Value;
    if (!db.Provinces.Any(p => p.ProvinceId == pid)) { error = "Please choose a valid province."; return null;
}


    var tn = LocationParser.Norm(town);
    var t = db.Towns.Where(x => x.ProvinceId == pid).AsEnumerable().FirstOrDefault(x =>
LocationParser.Norm(x.Name) == tn);
    if (t == null) { t = new Town { Name = town, ProvinceId = pid }; db.Towns.Add(t); db.SaveChanges(); }


    var sn = LocationParser.Norm(suburb);
    var sub = db.Suburbs.Where(x => x.TownId == t.TownId).AsEnumerable().FirstOrDefault(x =>
LocationParser.Norm(x.Name) == sn);
    if (sub == null) { sub = new Suburb { Name = suburb, TownId = t.TownId, PostalCode = postal };
db.Suburbs.Add(sub); db.SaveChanges(); }
    return sub.SuburbId;
}
```

*Figure 1.9 LocationHelper.ResolveOrCreate*

An address is stored as an address line plus a SuburbId, and the suburb leads to a town and a province. This avoids typing mistakes in the address and lets the system filter by place. The registration page and the patient and administrator profile pages show three linked drop-down boxes (Province, Town, Suburb) and the postal code fills in automatically. ResolveOrCreate turns whatever the page posted into one SuburbId to save.

If the person picked a suburb from the list, its id is posted, and the method only checks that the id exists. Otherwise the person chose "My suburb isn't listed" and typed the town and suburb, so the method validates the typed values first: a province, town and suburb must be present, the postal code must be exactly four digits, the names must not be longer than 100 characters and the province must exist. It then looks for the town in that province by comparing normalised names. LocationParser.Norm removes accents, spaces, hyphens and punctuation and converts to lower case, so "Jeffreys Bay", "jeffreys-bay" and "Jeffrey's Bay" are found as the same town and no duplicate is created. If the town is missing it is created, and the same is then done for the suburb with the postal code. The result is that new places can enter the database without an import, while the lists stay free of near-duplicates.

### 1.3.7 Creating a test request with unique sample barcodes (Doctor)

```
// The form shows one barcode per test, grouped by sample type. Hand them out per sample type in the
// order they were posted; anything missing, malformed or already used gets a fresh server-side barcode.
var posted = (model.Barcodes ?? new()).Where(b => b != null && !string.IsNullOrWhiteSpace(b.BarcodeNumber))
    .Select(b => new { Type = (b.SampleType ?? "").Trim(), Code = b.BarcodeNumber.Trim() }).ToList();
var used = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
var postedCodes = posted.Select(p => p.Code).Distinct().ToList();
var existing = new HashSet<string>(
    _ctx.RequestSamples.AsNoTracking().Where(x    =>    postedCodes.Contains(x.BarcodeLabel)).Select(x    =>
x.BarcodeLabel).ToList(),
    StringComparer.OrdinalIgnoreCase);
string TakeBarcode(string sampleTypeName)
{
    var      hit      =      posted.FirstOrDefault(p      =>      string.Equals(p.Type,      sampleTypeName,
StringComparison.OrdinalIgnoreCase)
        && p.Code.Length <= 40 && Regex.IsMatch(p.Code, @"^[A-Za-z0-9\-]+$")
        && !used.Contains(p.Code) && !existing.Contains(p.Code));
    if (hit != null) { used.Add(hit.Code); posted.Remove(hit); return hit.Code; }
    return NextBarcode(used);
}

foreach (var t in types.OrderBy(t => t.SampleType.Name).ThenBy(t => t.Name))
{
    var item = new TestRequestItem { TestTypeId = t.TestTypeId, TestRequestItemStatusId = 1 };
    item.RequestSamples.Add(new RequestSample
    {
        SampleTypeId = t.SampleTypeId,
        BarcodeLabel = TakeBarcode(t.SampleType.Name),
        CollectedOn = DateTime.UtcNow
    });
    request.TestRequestItems.Add(item);
}
_ctx.TestRequests.Add(request);
_ctx.SaveChanges();
EnsureTreatingDoctorConsent(patient.UserId, false, request.TestRequestItems);

private string NextBarcode(HashSet<string> usedNow)
{
    for (int i = 0; i < 20; i++)
    {
        var code = $"HF-{SastToday:yyMMdd}-{Random.Shared.Next(100000, 999999)}";
        if (usedNow.Contains(code) || _ctx.RequestSamples.Any(s => s.BarcodeLabel == code)) continue;
        usedNow.Add(code);
        return code;
    }
    throw new InvalidOperationException("Could not generate a unique barcode.");
}
```

*Figure 1.10 CreateTestRequest (barcode handling and saving) and NextBarcode*

When a doctor requests tests, every test needs a sample tube with a barcode label that is unique in the whole laboratory, otherwise two patients' tubes could be mixed up. The request form shows one barcode per test, grouped by sample type (the tube colour), and posts them with the form. The server cannot simply trust what the browser sends, so this code checks every barcode again before anything is saved.

The posted barcodes are first cleaned: blank entries are dropped and the rest is trimmed. One database query then loads every posted code that already exists in the RequestSample table into a case-insensitive HashSet, so the database is asked once and not once per barcode. The local function TakeBarcode hands out a barcode for a given sample type. It only accepts a posted code when the sample type matches, the code is at most 40 characters and contains only letters, digits and hyphens (a regular expression), the code has not already been given to another tube in this same request (the used set), and it does not exist in the database. When no valid code is left, NextBarcode creates one on the server in the format HF-yyMMdd-xxxxxx, using today's date in South African time and a six-digit random number.

It tries up to 20 times, checking both the codes already handed out in this request and the database, and throws an exception if it cannot find a free one.

The request is then built in memory. The tests are sorted by sample type and then by name so that tests on the same tube type appear together. For each test a TestRequestItem is created with status 1 (awaiting), and a RequestSample with its barcode is attached to the item. Because everything hangs off one TestRequest object, a single SaveChanges call inserts the request, all items and all samples in one transaction, so the database never contains a half-created request. Only after the save, when the database has generated the identity values, is EnsureTreatingDoctorConsent called. It makes sure the patient's consent page lists this doctor with access to only the tests that were just ordered, unless the doctor has already been given full access.

### 1.3.8 Loading patient medical data and enforcing consent restriction (Doctor)

```
async function loadPatientMedicalData(patient) {
    const container = document.getElementById('patientMedicalData');
    if (!patient) {
        container.style.display = 'none';
        return;
    }

    // Allergies, conditions, medication and previous results come from the database when a patient is chosen.
    if (!patient.loaded) {
        try {
            const r = await fetch('@(Url.Action("PatientSummary", "Doctor"))?id=' +
encodeURIComponent(patient.id), { headers: { 'Accept': 'application/json' } });
            if (r.ok) {
                const d = await r.json();
                patient.allergies = d.allergies || [];
                patient.conditions = d.conditions || [];
                patient.medications = d.medications || [];
                patient.hasGivenConsent = !!d.hasGivenConsent;
                patient.testHistory = d.testHistory || [];
                patient.loaded = true;
            }
        } catch (e) { console.error(e); }
    }
    // The doctor may have picked someone else while this was loading.
    if (document.getElementById('patientSelect').value !== String(patient.id)) return;

    renderBadges('allergyList', 'allergyCount', patient.allergies, 'warning');
    renderBadges('conditionList', 'conditionCount', patient.conditions, 'info');
    renderBadges('medicationList', 'medicationCount', patient.medications, 'success');
    renderTestHistory(patient);

    container.style.display = 'block';
}

function renderTestHistory(patient) {
    const countEl = document.getElementById('historyCount');
    const tbody = document.getElementById('historyTableBody');
    const consentWarning = document.getElementById('consentWarning');

    const filteredHistory = (patient.testHistory || []).filter(item =>
        patient.hasGivenConsent || item.isOrderedByCurrentDoctor
    );

    countEl.textContent = filteredHistory.length;
    consentWarning.style.display = patient.hasGivenConsent ? 'none' : 'block';

    if (filteredHistory.length === 0) {
        tbody.innerHTML = '<tr><td colspan="6" class="text-center text-muted small py-3">No previous test
results recorded for this patient.</td></tr>';
        return;
    }

    tbody.innerHTML = filteredHistory.map(item => {
        const valueBadge = item.isAbnormal
            ? `<span class="badge bg-danger-subtle text-danger border border-danger-
subtle">${item.resultValue}</span>`
```

```
            : `<span>${item.resultValue}</span>`;

        const doctorBadge = item.isOrderedByCurrentDoctor
            ? `${item.orderedByDoctor} <span class="badge bg-primary-subtle text-primary ms-1">You</span>`
            : item.orderedByDoctor;

        return `
            <tr>
                <td class="small">${item.orderDate}</td>
                <td class="fw-bold small">${item.testName}</td>
                <td class="small">${valueBadge}</td>
                <td class="text-muted small">${item.referenceRange}</td>
                <td class="small">${doctorBadge}</td>
                <td><span class="badge bg-success-subtle text-success">${item.status}</span></td>
            </tr>`;
    }).join('');
}
```

*Figure 1.11 loadPatientMedicalData and renderTestHistory (CreateTestRequest.cshtml)*

Before a doctor submits a new test request, they need to review the patient's allergies, conditions, medications, and previous test results to avoid duplicate orders or unsafe clinical decisions. At the same time, patient privacy rules require that tests ordered by other doctors remain hidden unless the patient has granted external consent.

When a doctor selects a patient from the dropdown or via the SA ID search, loadPatientMedicalData fetches that patient's medical summary asynchronously from the PatientSummary action on DoctorController and caches it on the client object (patient.loaded = true) so repeated selections do not trigger extra requests. To prevent race conditions if the doctor switches patients while a request is still in flight, the function verifies that the selected dropdown value still matches patient.id before updating the screen.

renderTestHistory then filters the returned testHistory array using patient.hasGivenConsent || item.isOrderedByCurrentDoctor. If the patient has not granted consent, any results ordered by other doctors are excluded, and the #consentWarning banner is displayed to explain why the view is restricted. Abnormal results are highlighted with a red badge, and tests ordered by the signed-in doctor are tagged with a You badge for quick identification.

### 1.3.9 Capturing and verifying a test result (Lab Technician)

```
var mineActive = item.TestRequestItemStatusId == ItTesting && item.AssignedTechnicianUserId == techUserId;
var mineReview = item.TestRequestItemStatusId == ItToBeReviewed && item.TestResult?.RecordedByUserId ==
techUserId;
if (!mineActive && !mineReview) return "You can only capture results for tests you selected.";
if (Math.Abs(value) >= 10_000_000m) return "That result value is too large.";
value = Math.Round(value, 3);


var tt = item.TestType;
var now = DateTime.UtcNow;
var res = item.TestResult;
if (res == null)
{
    res = new E.TestResult { TestRequestItemId = item.TestRequestItemId };
    _db.TestResults.Add(res);
}
res.ResultValue = value;
res.Unit = tt.Unit;
res.NormalMinAtTest = tt.NormalMin;
res.NormalMaxAtTest = tt.NormalMax;
res.RecordedByUserId = techUserId;
res.RecordedOn = now;
res.Notes = string.IsNullOrWhiteSpace(notes) ? null : (notes.Trim().Length > 500 ? notes.Trim()[..500] :
notes.Trim());


item.TestRequestItemStatusId = ItCompleted;
item.CompletedOn = now;


// Spec: once every test on the request has had its result captured (none left
// Awaiting/SampleCollected/Testing - a rejected sample doesn't block this), the
// request itself becomes Completed, independent of verification.
var outstanding = new[] { ItAwaiting, ItSampleCollected, ItTesting };
var allCaptured = item.TestRequest.TestRequestItems.All(i =>
!outstanding.Contains(i.TestRequestItemStatusId));
if (allCaptured && item.TestRequest.TestRequestStatusId == RqInProgress)
    item.TestRequest.TestRequestStatusId = RqCompleted;


_db.SaveChanges();


abnormal = (tt.NormalMin.HasValue && value < tt.NormalMin) || (tt.NormalMax.HasValue && value > tt.NormalMax);


// ... CaptureResult ends; Review begins ...


if (item.TestRequestItemStatusId != ItCompleted) return "That result is not waiting for verification.";
if (item.TestResult.RecordedByUserId == techUserId) return "You can't verify your own result.";
if (!_db.TechnicianTestTypes.Any(t => t.TechnicianUserId == techUserId && t.TestTypeId == item.TestTypeId))
    return "That test type hasn't been assigned to you by the laboratory manager.";


var res = item.TestResult;
_db.Verifications.Add(new E.Verification
{
    TestResultId = res.TestResultId,
    VerifiedByUserId = techUserId,
    VerifiedOn = DateTime.UtcNow,
    Comment = string.IsNullOrWhiteSpace(note) ? null : (note.Trim().Length > 500 ? note.Trim()[..500] :
note.Trim()),
    Outcome = verify ? "Verified" : "Returned",
    ResultValue = res.ResultValue,
    ResultRecordedByUserId = res.RecordedByUserId,
    ResultRecordedOn = res.RecordedOn
});
item.TestRequestItemStatusId = verify ? ItVerified : ItToBeReviewed;


if (verify)
{
    var all = item.TestRequest.TestRequestItems;
    allVerified = all.All(i => i.TestRequestItemStatusId == ItVerified);   // this item was just set above
```

*Figure 1.12 LabTechService.CaptureResult and LabTechService.Review*

A laboratory result must be traceable and must be checked by a second person. The two methods above implement this. CaptureResult runs when a technician enters a value. It first checks that the technician is allowed to do so: the test must be assigned to this technician and currently in the Testing state, or it must be a result that was returned to this technician for review. The value is limited to a sensible size and rounded to three decimals.

The result row is created, or overwritten when the result is recaptured after a review. The unit and the normal minimum and maximum are copied from the test type into the result (NormalMinAtTest and NormalMaxAtTest). This is deliberate: if the laboratory manager later changes the normal range for a test, results that were already captured still show the range that applied on the day. The test item is set to Completed with the time. The request becomes Completed as soon as no test on it is still waiting, collected or being tested; a rejected sample does not block this, and the rule is independent of verification. The method also returns an abnormal flag, which is true when the value is below the minimum or above the maximum, so the screen and the activity log can warn the technician.

Review is the second step, verification. It enforces the separation of duties from the specification: it refuses if the result is not waiting for verification, if the technician is the one who captured it ("You can't verify your own result"), or if the test type has not been assigned to this technician by the laboratory manager. A Verification row is then written that keeps a snapshot of the value, who recorded it and when, together with the outcome (Verified or Returned) and the comment, so the audit trail stays correct even if the result is recaptured later. A verified item becomes Verified and a returned item goes back to the original technician with the status To be reviewed. When the item just verified was the last one, allVerified becomes true, which is the trigger for the controller to e-mail the requesting doctor the results as a PDF.

### 1.3.10 Receiving samples and rejecting unusable ones (Lab Technician)

```
public string? ReceiveSamples(int requestId, IEnumerable<LM.ReceivedSampleInput> samples, int techUserId)
{
    var req = _db.TestRequests.Include(r => r.TestRequestItems).ThenInclude(i => i.RequestSamples)
        .FirstOrDefault(r => r.TestRequestId == requestId);
    if (req == null) return "That request was not found.";
    if (req.TestRequestStatusId != RqSubmitted && req.TestRequestStatusId != RqSamplesReceived)
        return "That request has no samples waiting to be received.";
    var waiting = req.TestRequestItems.Where(i => i.TestRequestItemStatusId == ItAwaiting).ToList();
    if (!waiting.Any()) return "That request has no samples waiting to be received.";


    var now = DateTime.UtcNow;
    foreach (var s in samples.ToList())
    {
        var original = (s.OriginalBarcode ?? "").Trim();
        var sample = waiting.SelectMany(i => i.RequestSamples)
            .FirstOrDefault(x => string.Equals(x.BarcodeLabel, original, StringComparison.OrdinalIgnoreCase));
        if (sample == null) return $"Barcode '{original}' does not belong to this request.";


        if (string.Equals(s.Action, "Reject", StringComparison.OrdinalIgnoreCase))
        {
            var reason = (s.RejectReason ?? "").Trim();
            if (reason.Length == 0) return $"A reason is required to reject the sample '{original}'.";
            sample.RejectReason = reason.Length > 500 ? reason[..500] : reason;
            sample.RejectedOn = now;
            sample.RejectedByUserId = techUserId;
            sample.TestRequestItem.TestRequestItemStatusId = ItRejected;
            continue;
        }


        var barcode = (s.Barcode ?? "").Trim();
        if (!BarcodeRx.IsMatch(barcode)) return $"Barcode '{barcode}' is not valid (letters, numbers and
hyphens, up to 40 characters).";
        if (!string.Equals(sample.BarcodeLabel, barcode, StringComparison.Ordinal))
        {
            if (_db.RequestSamples.Any(x => x.RequestSampleId != sample.RequestSampleId && x.BarcodeLabel ==
barcode))
                return $"Barcode '{barcode}' is already used by another sample.";
            sample.BarcodeLabel = barcode;
        }
        sample.CollectedByUserId = techUserId;
        sample.ReceivedOn = now;
        sample.TestRequestItem.TestRequestItemStatusId = ItSampleCollected;
    }
    if (req.TestRequestStatusId == RqSubmitted) req.TestRequestStatusId = RqSamplesReceived;
    _db.SaveChanges();
    return null;
}
```

*Figure 1.13 LabTechService.ReceiveSamples*

When the samples for a test request arrive at the laboratory, a technician has to record that they were received, by whom and when, and confirm the barcode on each tube. The specification also requires the request to move to the status Samples Received. In practice not every tube is usable: a sample can be clotted, haemolysed or in the wrong tube. The first version of the page only allowed the whole request to be accepted, so I changed it so that every sample on the request is accepted or rejected on its own. ReceiveSamples is the method that saves that decision.

The request is loaded once with all its test items and their samples (Include and ThenInclude), so the loop below works in memory and does not query the database for every tube. The method refuses to continue if the request is not in Submitted or Samples Received, or if none of its items are still awaiting a sample, which stops a request from being received twice when two technicians open it at the same time.

Each posted sample is matched to the request by its original barcode, so a form that was changed by hand cannot touch a tube that belongs to another request. A rejected sample must have a reason; the reason, the time in UTC and the technician are stored on the RequestSample row and the test item becomes Rejected. An accepted sample may have its barcode corrected if the printed label was wrong. The new barcode is checked against a regular expression and against every other sample in the database, because a duplicate barcode could mix up two patients' tubes. The receiving technician and time are then saved and the item becomes Sample Collected, which makes it available for selection.

Finally the request itself moves from Submitted to Samples Received, and everything is written in one SaveChanges call, so a validation error part-way through leaves nothing half-saved. When samples were rejected, the controller e-mails the requesting doctor with the barcode, the tests on that tube and the reason, so the doctor can arrange a new sample while the remaining tests continue.

### 1.3.11 Selecting a test and deducting consumables from stock (Lab Technician)

```
// "OutOfStock" if any consumable the test needs has less on hand than one run uses,
// "LowStock" if one run takes a consumable to or under its reorder level, else "OK".
public string StockStatus(int testTypeId)
{
    EnsureCatalogue();
    if (!_types!.TryGetValue(testTypeId, out var tt) || tt.TestTypeConsumables == null ||
!tt.TestTypeConsumables.Any()) return "OK";
    var status = "OK";
    foreach (var link in tt.TestTypeConsumables)
    {
        if (!_consumables!.TryGetValue(link.ConsumableId, out var c)) continue;
        if (c.QuantityOnHand < link.QuantityRequired) return "OutOfStock";
        if (c.QuantityOnHand - link.QuantityRequired <= c.ReorderLevel) status = "LowStock";
    }
    return status;
}

public string? SelectTest(int itemId, int techUserId)
{
    var item = _db.TestRequestItems.Include(i => i.TestRequest).FirstOrDefault(i => i.TestRequestItemId ==
itemId);
    if (item == null || item.TestRequestItemStatusId != ItSampleCollected ||
item.TestRequest.TestRequestStatusId == RqCancelled)
        return "That test is no longer waiting to be selected.";
    if (!_db.TechnicianTestTypes.Any(t => t.TechnicianUserId == techUserId && t.TestTypeId == item.TestTypeId))
        return "That test type hasn't been assigned to you by the laboratory manager.";

    // Consumables the test needs: refuse if short, otherwise deduct (same save, so both or neither).
    var needs = _db.TestConsumables.Where(l => l.TestTypeId == item.TestTypeId).ToList();
    var cons = _db.Consumables.Where(c => needs.Select(n => n.ConsumableId).Contains(c.ConsumableId)).ToList();
    foreach (var n in needs)
    {
        var c = cons.FirstOrDefault(x => x.ConsumableId == n.ConsumableId);
        if (c != null && c.QuantityOnHand < n.QuantityPerTest)
            return $"Not enough {c.Name} in stock to perform this test.";
    }
    foreach (var n in needs)
    {
        var c = cons.FirstOrDefault(x => x.ConsumableId == n.ConsumableId);
        if (c != null) c.QuantityOnHand -= n.QuantityPerTest;
    }

    item.TestRequestItemStatusId = ItTesting;
    item.AssignedTechnicianUserId = techUserId;
    item.StartedOn = DateTime.UtcNow;
    if (item.TestRequest.TestRequestStatusId == RqSubmitted || item.TestRequest.TestRequestStatusId ==
RqSamplesReceived)
        item.TestRequest.TestRequestStatusId = RqInProgress;
    _db.SaveChanges();
    return null;
}
```

*Figure 1.14 LabTechService.StockStatus and LabTechService.SelectTest*

After a sample is received a technician selects the test to perform it. The specification sets two rules here: a technician may only select test types that the laboratory manager has assigned to them, and the consumables that a test uses must be deducted from stock automatically. These two methods implement both rules.

StockStatus is used by the Dashboard and the Select Test page to show a stock badge on every test before the technician picks it. For each consumable linked to the test type (the TestConsumable table, with the quantity used per test) it compares the quantity on hand with the quantity one test needs. If any consumable is short the test is Out of Stock and the Select button is disabled; if one test would bring a consumable down to or below its reorder level it is shown as Low Stock, so the technician can warn the laboratory manager before it runs

out. The catalogue of test types and consumables is loaded once per request by EnsureCatalogue, so the badges for a whole page do not cause one query per test.

SelectTest is the action itself. It checks again on the server that the test is still waiting to be selected and has not been cancelled, and that the test type is linked to this technician in TechnicianTestType, because a hidden button is not security and a form can be posted by hand. It then loads the consumables the test uses and checks all of them first before changing any of them: if the check and the deduction were done in one loop, a shortage on the second consumable would leave the first one already reduced. Only when every consumable is available are the quantities deducted.

The test item becomes Testing, is assigned to the technician with the start time, and the request moves to In Progress the first time one of its tests is started. The stock deduction and the status change are saved by the same SaveChanges call, so either both happen or neither does.

### 1.3.12 Building the technician dashboard and the turnaround time rules (Lab Technician)

```
// QueueItem: the turnaround clock starts when the sample is received.
public DateTime? DueAt => ReceivedAt?.AddMinutes(TestType?.TurnaroundTime ?? 0);
private bool TatRunning => Status is "Available" or "Active" or "Review";
public bool IsOverdue => TatRunning && DueAt.HasValue && DueAt.Value < SastNow;
public bool IsNearingTat => TatRunning && DueAt.HasValue && !IsOverdue
                            && (DueAt.Value - SastNow).TotalMinutes <= NearingTatMinutes;


// LabTechController.Dashboard
var filter = new LabTechFilter { Urgency = urgency, Category = category, Due = due, Req = req };
var permitted = PermittedTestTypeIds;
var visible = filter.Apply(Queue.Where(q => q.Status != "Cancelled")).ToList();


var mine = visible.Where(q => q.Status == "Active" && q.AssignedTechnicianId ==
CurrentTechnicianId).ToList();
var available = visible.Where(q => q.Status == "Available" && permitted.Contains(q.TestTypeId)).ToList();
var toVerify  = visible.Where(CanVerify).ToList();
var toReview  = visible.Where(IsMyReview).ToList();
var workload  = mine.Concat(available).Concat(toReview).ToList();


var lists = new List<LabTechQueueList>
{
    new() { Key = "stat",     Title = "Urgent (STAT)",              Items = workload.Where(q => q.Urgency ==
"STAT").ToList() },
    new() { Key = "overdue", Title = "Overdue",               Items = workload.Where(q => q.IsOverdue).ToList()
},
    new() { Key = "nearing", Title = "Nearing Turnaround Limit", Items = workload.Where(q =>
q.IsNearingTat).ToList() },
    new() { Key = "mine",    Title = "My Tests",                Items = mine },
    new() { Key = "available", Title = "Waiting to be Selected", Items = available },
    new() { Key = "verify",  Title = "Waiting to be Verified",   Items = toVerify },
    new() { Key = "review",  Title = "Waiting to be Reviewed",   Items = toReview },
};
foreach (var l in lists)
    l.Items = l.Items.OrderBy(q => q.DueAt ?? DateTime.MaxValue).ToList();


// A technician cannot verify their own result, only test types assigned to them.
private bool CanVerify(QueueItem q) =>
    q.Status == "Verify" && q.CapturedByUserId != CurrentTechnicianId &&
PermittedTestTypeIds.Contains(q.TestTypeId);
```

*Figure 1.15 QueueItem turnaround properties and LabTechController.Dashboard (extract)*

The specification asks for a technician dashboard that shows the tests selected by the technician, tests waiting to be selected, waiting to be verified and waiting to be reviewed, and that highlights STAT tests, overdue tests and tests close to their turnaround time limit. The dashboard must also be filterable. The code above decides which tests appear in each of these lists.

The turnaround rules are properties of QueueItem, the view model that LabTechService builds from the TestRequestItem, RequestSample, TestResult and Verification tables. DueAt is the time the sample was received plus the turnaround time of the test type that the laboratory manager set. The clock only runs while the result is still outstanding (Available, Active or returned for Review): once a result is captured it no longer counts as overdue. IsOverdue is true when DueAt has passed, and IsNearingTat when the test is not yet overdue but is due within the next 30 minutes. Both compare against the current South African time, because ReceivedAt was converted from UTC to SAST when the view model was built.

In Dashboard the filters chosen by the technician (urgency, test category, due status and request number) are applied first, so every card on the page shows the same filtered set. The visible tests are then split by the rules of the specification: My Tests are the active tests assigned to this technician; Waiting to be Selected only shows received samples for test

types assigned to this technician; Waiting to be Verified uses CanVerify, which excludes results this technician captured and test types they are not assigned to; and Waiting to be Reviewed only shows this technician's own results that a verifier returned. The STAT, Overdue and Nearing cards are built from the technician's own workload rather than from every test in the laboratory, so a technician is not alerted about work they cannot perform. Every list is sorted by DueAt so the most urgent test is always at the top, and tests without a due time are placed last.

### 1.3.13 Receiving stock and recalculating the order status (Lab Manager)

```
if (!ItemIsOpen(item))
{
    TempData["WarningMessage"] = "This item has already been received or cancelled.";
    return RedirectToAction("Details", new { id = orderId });
}

item.OrderItemStatusId = await ItemStatusId("Received");
item.OrderItemStatus = await _context.OrderItemStatuses.SingleAsync(s => s.Code == "Received");
item.QuantityReceived = item.QuantityOrdered;
item.ReceivedOn = NowUtc;
item.Consumable.QuantityOnHand += item.QuantityOrdered;     // stock updated automatically

await RecalculateOrderStatus(order);
await _context.SaveChangesAsync();                          // one save = all or nothing

private async Task RecalculateOrderStatus(DbOrder order)
{
    var codes = order.OrderItems.Select(i => i.OrderItemStatus.Code).ToList();

    if (codes.All(c => c == "Cancelled"))
    {
        order.OrderStatusId = await OrderStatusId("Cancelled");
        order.CancelledOn ??= NowUtc;
        order.CancelReason ??= "All items cancelled";
    }
    else if (codes.All(c => c == "Received" || c == "Cancelled"))
    {
        order.OrderStatusId = await OrderStatusId("Received");
        order.ReceivedOn ??= NowUtc;
    }
    else if (codes.Any(c => c == "Received"))
    {
        order.OrderStatusId = await OrderStatusId("PartiallyReceived");
    }
}
```

*Figure 1.16 OrderController.ReceiveItem and RecalculateOrderStatus*

When a supplier delivers consumables, the lab manager marks each order item as received. The stock level in the Consumable table then has to go up by the quantity delivered, and the status of the whole order has to follow from the status of its items. Both must happen together, otherwise the stock and the order would disagree.

ReceiveItem first loads the order with its supplier, status, items, consumables and item statuses in one query (LoadOrderForUpdate uses Include and ThenInclude). It then refuses the action when the item is no longer open, that is already received or cancelled. This guard matters because a double click or a second browser tab would otherwise add the same delivery to the stock twice. For an open item the status is set to Received, the quantity received is set to the quantity ordered, the time is recorded in UTC, and the stock on hand

of the consumable is increased by the quantity ordered. The stock figure is therefore updated automatically, and the manager does not type it in.

RecalculateOrderStatus then applies three rules to the status codes of all the items on the order. If every item is cancelled the order becomes Cancelled and the cancel date and a reason are filled in. If every item is either received or cancelled the order becomes Received and the received date is filled in. If at least one item has been received but others are still open, the order becomes Partially received. The operator ??= is used for the dates so that a date that was set earlier is never overwritten. Only after the status has been recalculated is one SaveChangesAsync called, so the item status, the stock level and the order status are saved together or not at all. The action is written to the activity log after the save, showing the new stock level.

### 1.3.14 Building the "Tests performed by category" report (Lab Manager)

```
// South African calendar dates -> UTC range (RecordedOn is stored in UTC)
var fromUtc = SastTime.SastDateToUtc(startDate.Date);
var toUtc   = SastTime.SastDateToUtc(endDate.Date.AddDays(1));

// 1. One database query: tests performed and abnormal results per test type
var perType = await _context.TestResults.AsNoTracking()
    .Where(r => r.RecordedOn >= fromUtc && r.RecordedOn < toUtc)
    .GroupBy(r => r.TestRequestItem.TestTypeId)
    .Select(g => new
    {
        TestTypeId = g.Key,
        Total      = g.Count(),
        Abnormal   = g.Sum(r =>
            (r.NormalMinAtTest != null && r.ResultValue < r.NormalMinAtTest) ||
            (r.NormalMaxAtTest != null && r.ResultValue > r.NormalMaxAtTest) ? 1 : 0)
    })
    .ToListAsync();
var countedTypeIds = perType.Select(x => x.TestTypeId).ToList();

// 2. Active test types + archived ones that still had tests in the period
var testTypes = await _context.TestTypes.AsNoTracking()
    .Where(t => !t.IsArchived || countedTypeIds.Contains(t.TestTypeId))
    .Select(t => new { t.TestTypeId, t.Name, t.IsArchived, t.TestCategoryId,
                       SampleType = t.SampleType.Name })
    .ToListAsync();
var categoryIdsWithTypes = testTypes.Where(t => countedTypeIds.Contains(t.TestTypeId))
    .Select(t => t.TestCategoryId).Distinct().ToList();

var categories = await _context.TestCategories.AsNoTracking()
    .Where(c => !c.IsArchived || categoryIdsWithTypes.Contains(c.TestCategoryId))
    .Select(c => new { c.TestCategoryId, c.Name, c.IsArchived })
    .ToListAsync();

// 3. Roll the test type figures up into their categories (in memory)
return categories.Select(c =>
{
    var types = testTypes.Where(t => t.TestCategoryId == c.TestCategoryId)
        .Select(t =>
        {
            var n = perType.FirstOrDefault(x => x.TestTypeId == t.TestTypeId);
            return new LabManagerTestTypeReportRow
            {
                TestType = t.Name, SampleType = t.SampleType, IsArchived = t.IsArchived,
                TotalTests = n?.Total ?? 0, AbnormalTests = n?.Abnormal ?? 0
            };
        })
        .Where(t => !t.IsArchived || t.TotalTests > 0)
        .OrderByDescending(t => t.TotalTests).ThenBy(t => t.TestType)
        .ToList();

    return new LabManagerCategoryReportRow
    {
        Category = c.Name, IsArchived = c.IsArchived, TestTypes = types,
```

```
        TotalTests = types.Sum(t => t.TotalTests), AbnormalTests = types.Sum(t => t.AbnormalTests)
    };
})
.OrderByDescending(c => c.TotalTests).ThenBy(c => c.Category)
.ToList();
```

*Figure 1.17 ReportController.BuildReport*

The specification asks the Lab Manager for a PDF report of the total tests performed in a date range, grouped by category. A test counts as performed when its result has been captured, so the report is built from the TestResult table. The difficulty is that a result does not know its category directly: it belongs to a test request item, which points to a test type, which belongs to a category. The report also has to show test types and categories that had no tests, and it must not lose results for test types or categories that were archived after the tests were done.

The method first turns the two dates that the manager picks into a UTC range. The dates are South African calendar days, but RecordedOn is stored in UTC, so SastTime.SastDateToUtc converts the start of the first day and the start of the day after the last day. Using "less than the next day" instead of "less than or equal to the last day" makes sure that results captured late on the last day are included, and the report is correct even when the server runs in another time zone.

Step 1 is a single grouped query that runs in SQL Server. It groups the results in the range by test type and returns, for each test type, the number of tests and the number of abnormal results. A result is abnormal when its value is below NormalMinAtTest or above NormalMaxAtTest. These are the reference limits that were copied onto the result when it was captured, so changing a test type's range later does not change old reports. The conditional Sum counts the abnormal results in the database instead of loading every result into memory.

Step 2 loads the test types and categories to report on: every active one, plus any archived one that still had tests in the period. Without the second part, archiving a category would make its past tests disappear from the totals. Step 3 then joins the counts to the test types in memory. A test type without results gets 0 through the ?? operator, an archived test type is only kept when it was used, and each category's total and abnormal count are the sums of its test types. Both lists are sorted by tests performed, highest first, then by name. The result is passed to the PDF service, which prints a summary by category and a breakdown table per category with subtotals.

### 1.3.15 Assigning and removing technicians from a test type (Lab Manager)

```
// AssignTechnicians: only real, active technicians not yet linked to this test type
var toAdd = await _context.TechnicianProfiles
    .Where(p => wanted.Contains(p.UserId) && !p.IsArchived && !p.User.IsArchived
             && !p.TechnicianTestTypes.Any(x => x.TestTypeId == id))
    .Select(p => new { p.UserId, Name = p.User.FirstName + " " + p.User.Surname })
    .ToListAsync();

foreach (var t in toAdd)
    _context.TechnicianTestTypes.Add(
        new TechnicianTestType { TechnicianUserId = t.UserId, TestTypeId = id });
await _context.SaveChangesAsync();

// RemoveTechnician: every technician must keep at least one active test type
var link = await _context.TechnicianTestTypes
    .Include(x => x.TechnicianUser).ThenInclude(p => p.User)
    .FirstOrDefaultAsync(x => x.TestTypeId == id && x.TechnicianUserId == technicianId);
if (link == null) return NotFound();

var name = $"{link.TechnicianUser.User.FirstName} {link.TechnicianUser.User.Surname}";

int otherActiveTypes = await _context.TechnicianTestTypes
    .CountAsync(x => x.TechnicianUserId == technicianId && x.TestTypeId != id
                  && !x.TestType.IsArchived);
if (otherActiveTypes == 0)
{
    TempData["ErrorMessage"] = $"{name} can't be removed - this is their only test type. " +
                               "Assign them another test type first.";
    return RedirectToAction("Technicians", new { id });
}

_context.TechnicianTestTypes.Remove(link);
await _context.SaveChangesAsync();
await Activity.LogAsync("Unassigned", "TestType", testType.Name,
                        $"Technician {name} removed from this test");
```

*Figure 1.18 TestTypeController.AssignTechnicians and RemoveTechnician (extract)*

The link between technicians and the tests they may perform is the TechnicianTestType table. The specification lets the Lab Manager see which technicians can perform a test type, and after a demonstration the lecturer asked that technicians can also be assigned and removed directly from that page instead of only from each technician's own Edit page. Both actions change the same link table, so they had to keep the same rules as the technician pages.

AssignTechnicians receives the list of ticked technician IDs from the form. The posted values are never trusted directly: the query only returns technicians whose IDs were ticked, whose profile and user account are not archived, and who are not already linked to this test type (the Any condition inside the Where). This means a technician can never be linked twice, even when the form is submitted twice or edited in the browser. A TechnicianTestType row is added for each remaining technician and all of them are saved with one SaveChangesAsync call, so either all are assigned or none.

RemoveTechnician enforces the rule that every technician must be able to perform at least one test type. It loads the link together with the technician's user details (Include and ThenInclude), and then counts the technician's other links to test types that are not archived. Links to archived test types are not counted, because the technician can no longer perform those tests. If there are no other active links, the link is not removed and the manager is told to assign another test type first. Otherwise the row is deleted, saved and written to the activity log, and the page shows a confirmation message.

## 1.4 Source code references

The system uses the following code, data and styling that were created by other authors. None of it is listed in full. For every item the reason for using it, where it is used and an APA reference are given.

### 1.4.1 Bootstrap (layout, forms, modals, dropdowns)

***Description***

Bootstrap is a CSS and JavaScript framework. It was used so that every page has a responsive layout and consistent forms, buttons, modals and drop-down menus without writing all of the CSS by hand. Version 5.1.0 is stored in the project.

***Usage***

Every page of every portal, for example the Register page, the Admin User Management page and the Patient Medical Profile page.

***Reference***

Bootstrap Team. (2021). Bootstrap (Version 5.1.0) [Computer software]. https://getbootstrap.com/

### 1.4.2 Bootstrap Icons

***Description***

Bootstrap Icons provides the small icons used in the sidebars, buttons, status labels and messages.

***Usage***

The sidebar menu of every portal and the buttons on the Admin and Patient pages.

***Reference***

Bootstrap Team. (2023). Bootstrap Icons (Version 1.11.1) [Computer software]. https://icons.getbootstrap.com/

### 1.4.3 Drawing the results graph

***Description***

Chart.js draws the line chart that shows one test over time with the normal range and the median. It was used because it produces an interactive responsive chart from a small amount of JavaScript.

***Usage***

Patient portal: Track Results page (and the trend charts on the doctor pages).

***Reference***

Chart.js Contributors. (2025). Chart.js (Version 4.5.1) [Computer software]. https://www.chartjs.org/

### 1.4.4 Showing the two-factor QR code

***Description***

QRCode.js turns the otpauth link into a QR code in the browser so an authenticator app can scan it.

***Usage***

Patient, Administrator and Lab Technician portals: Set Up Two-Factor Authentication page.

***Reference***

Shim, D. (n.d.). QRCode.js (Version 1.0.0) [Computer software]. GitHub. https://github.com/davidshimjs/qrcodejs

### 1.4.5 Page and menu text font

***Description***

The Inter typeface is used for all text so the pages are easy to read and look the same on every computer. The font files are stored in the project.

***Usage***

All pages of all portals.

***Reference***

Andersson, R. (n.d.). Inter [Typeface]. https://rsms.me/inter/

### 1.4.6 Brand name font

***Description***

The Sora typeface is used for the HemaFlow name on the public pages to give the brand its own look.

***Usage***

Public pages: the navigation bar and the home page.

***Reference***

Sora Project Authors. (n.d.). Sora [Typeface]. Google Fonts. https://fonts.google.com/specimen/Sora

### 1.4.7 South African suburb and postal code list

***Description***

GeoNames postal code data for South Africa was used to build the Province, Town and Suburb lists behind the address drop-down boxes. The data is licensed under Creative Commons Attribution 4.0 and is credited here as required.

***Usage***

Register page, Patient profile (Edit Profile) and Administrator profile: the address boxes.

***Reference***

GeoNames. (n.d.). GeoNames postal code data: South Africa (ZA) [Data set]. Creative Commons Attribution 4.0. https://download.geonames.org/export/zip/

### 1.4.8 Time-based one-time password algorithm

***Description***

The TOTP algorithm is described in RFC 6238 and builds on the HOTP algorithm in RFC 4226. The Base32 text format used for the secret is described in RFC 4648. The code in TotpService follows these standards so that standard authenticator apps give matching codes. The code itself was written for the project.

***Usage***

Patient, Administrator and Lab Technician portals: Set Up Two-Factor Authentication, and the two-factor step at sign-in.

***Reference***

M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). TOTP: Time-based one-time password algorithm (RFC 6238). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc6238

M'Raihi, D., Bellare, M., Hoornaert, F., Naccache, D., & Ranen, O. (2005). HOTP: An HMAC-based one-time password algorithm (RFC 4226). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc4226

Josefsson, S. (2006). The Base16, Base32, and Base64 data encodings (RFC 4648). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc4648

### 1.4.9 Creating the PDF reports

***Description***

QuestPDF is the library that lays out and renders the PDF reports. It was used because it describes a page in C# code, which made it possible to give every role the same header, footer and layout.

***Usage***

Patient portal: My Reports and the result downloads; Administrator portal: the Export PDF buttons.

***Reference***

QuestPDF. (2024). QuestPDF (Version 2024.3.6) [Computer software]. https://www.questpdf.com/

### 1.4.10 Pop-up notifications (toasts)

***Description***

The notification style that slides in at the top right of the page (success, warning and error messages with a progress bar) is based on a design from Uiverse, a library of open-source user interface elements. We adapted the design and rewrote it as our own HemaUI helper script.

***Usage***

Every portal: success and error messages, for example after saving a profile or approving a request.

***Reference***

Uiverse. (n.d.). Uiverse: Open-source UI elements for any project [Website]. https://uiverse.io/

### 1.4.11 South African ID number check digit

***Description***

A South African ID number has 13 digits: a birth date, a gender sequence, a citizenship digit and a final check digit that is calculated with the Luhn algorithm. We implemented the check in our own validation attribute so that the Register page rejects numbers that cannot be real. The validation follows the published format of the ID number and the Luhn algorithm.

***Usage***

Patient portal: Register page (server-side validation of the ID number).

***Reference***

Luhn, H. P. (1960). Computer for verifying numbers (U.S. Patent No. 2,950,048). United States Patent Office.

Department of Home Affairs. (n.d.). Identity document: South African identity number structure. https://www.dha.gov.za/

### 1.4.12 South African ID number check for Lab Technicians (Lab Manager)

***Description***

The specification requires a unique South African ID number for every Lab Technician. I wrote a validation attribute (SouthAfricanIdAttribute) that checks that the number has 13 digits, that the first six digits are a real date of birth, that the citizenship digit is 0 or 1, and that the last digit matches the Luhn check digit. The check follows the published structure of the ID number and the Luhn algorithm.

***Usage***

Lab Manager portal: Add Technician and Edit Technician pages.

***Reference***

Luhn, H. P. (1960). Computer for verifying numbers (U.S. Patent No. 2,950,048). United States Patent Office.

Department of Home Affairs. (n.d.). Identity document: South African identity number structure. https://www.dha.gov.za/

### 1.4.13 Lab Manager PDF report (Lab Manager)

***Description***

The "Tests performed by category" report is laid out with QuestPDF and uses the shared HemaFlow header and footer. The tables, subtotals and notes of the Lab Manager report were written with the QuestPDF fluent API, following the QuestPDF documentation.

***Usage***

Lab Manager portal: Reports page, Generate PDF.

***Reference***

QuestPDF. (2024). QuestPDF (Version 2024.3.6) [Computer software]. https://www.questpdf.com/

### 1.4.14 Weather on the technician dashboard (Lab Technician)

#### *Description*

The welcome strip at the top of the Lab Technician dashboard shows the current temperature, weather and wind speed in Gqeberha next to the date and time. The data comes from the free Open-Meteo weather API, which needs no key. The data is licensed under Creative Commons Attribution 4.0 and is credited here as required.

#### *Usage*

Lab Technician portal: Dashboard (welcome strip).

#### *Reference*

Open-Meteo. (n.d.). Open-Meteo free weather API [Computer software]. Creative Commons Attribution 4.0. https://open-meteo.com/

### 1.4.15 Technician report and doctor results PDF (Lab Technician)

#### *Description*

The "Tests completed by technician" report and the results PDF that is e-mailed to the doctor when all tests on a request are verified are laid out with QuestPDF and use the shared HemaFlow header and footer. The report tables use the QuestPDF Table element with a header row, so the column headings repeat on every page of a long report, following the QuestPDF documentation.

#### *Usage*

Lab Technician portal: Technician Report page (Preview, Download PDF and Email Me a Copy), and the results e-mail sent to the doctor after verification.

#### *Reference*

QuestPDF. (2024). QuestPDF (Version 2024.3.6) [Computer software]. https://www.questpdf.com/

## 1.5 AI Used

### 1.5.1 Gemini (Google)

Gemini, an AI assistant made by Google, was used to help with the pre-loading of data into the system as well as supporting the development of the Doctor subsystem. For data pre-loading, we checked the generated sample data (such as master lists and example records) and loaded it into the database with our SQL scripts. For the Doctor subsystem, Gemini was used for:

- Standardising the user interface styling across the Doctor portal views (cards, buttons, and status badges) using Bootstrap.
- Assisting with Razor view and ViewModel hookups (such as aligning ViewResults.cshtml and CreateTestRequest.cshtml with their respective ViewModels) and verifying SA ID checksums for test data.
- Reviewing database context configuration and helping structure the technical documentation for the Doctor subsystem.

### 1.5.2 Claude (Anthropic)

Claude, an AI assistant made by Anthropic, was used for support while building the Admin, Patient, Lab Manager and Lab Technician subsystems. Whenever we got stuck, we asked Claude for assistance and it helped us find and fix the problem, and we also asked for feedback on ways to improve the system. We always reviewed the suggested code and scripts, added them to the project, built and tested them ourselves in Visual Studio against our database, and only then committed them. Claude was used for:

- Adding short comments to the code so that it is easier to read.
- Helping with the user interface, for example layout, spacing, and the look of pages and messages.
- Helping with the controllers and making sure that the screens were linked correctly to the database.
- Moving the Lab Manager pages from temporary in-memory lists to the shared SQL Server database, including the Entity Framework queries in the controllers.
- Writing the SQL scripts for the Lab Manager tables (haematology preload data for judging) and read-only scripts to check the data.
- Lab Technician subsystem: restyling the technician pages to match the look of the Doctor portal, including the dashboard cards, the pop-up windows and the review history stepper.
- Lab Technician subsystem: helping to find the cause of the problems described in 1.6.12 to 1.6.15, and checking the technician pages against the Lab Technician requirements in the ONP400 specification.
- Lab Technician subsystem: the layout of the technician PDF report and the doctor results e-mail.

## 1.6 Problems encountered

### 1.6.1 We built the database on the wrong version of SQL Server (Admin & Patient)

We originally built our database on SQL Server 2025, because that was the version on our own computers. The campus server runs SQL Server 2022, and a database cannot be moved from a newer version of SQL Server to an older one, so the database could not be attached on the server.

We had no choice but to start the database again on SQL Server 2022. Because our tables and seed data are created by numbered SQL scripts, we could rebuild the whole database by running the scripts in order. We learned to check the version of the server we are delivering to before starting to build, and to keep every change in a script.

### 1.6.2 The site showed "HTTP Error 500.30" after publishing (Admin & Patient)

After we published the application to the campus IIS site, every page returned HTTP Error 500.30. This error only says that the application failed to start, and the page gave no useful reason.

We first looked for a mistake in our code and settings, but the same build ran correctly on our own computers. The problem turned out to be on the server side: we had not uploaded the whole published version, so files that the application needs to start were missing from the IIS folder. After we copied the complete publish folder to the server again, the site started. We now always copy the entire publish output in one go and check that the file list on the server matches the publish folder.

### 1.6.3 "That user could not be found" when resetting a password (Admin)

While testing User Management as a normal Administrator we changed the surname of a technician and then pressed Reset Password. The page answered "That user could not be found", although the user was in the list. We first thought the lookup code was wrong.

We looked at the Users table in SSMS and found that many seeded accounts, including the technician, had IsArchived set to 1. The list was showing archived accounts as if they were normal, while the actions correctly refused to work on them. We fixed the design instead of the lookup: archived users now show the status Archived and are moved to their own Archived page, they have no Manage menu, and the server returns a clear message if someone still tries to change one.

### 1.6.4 The browser Back button showed private pages after logging out (Admin)

After we logged out as an administrator and pressed the Back button, the browser still showed the dashboard. The data on the page was the old copy that the browser had kept in its cache.

We added a small piece of middleware in Program.cs that sends Cache-Control: no-store (together with the Pragma and Expires headers) for every page that a signed-in user receives. After logging out, the Back button now leads to the login page instead of the old page. Testing this showed us that a hidden button or a redirect is not enough, because the browser can show a stored page without asking the server.

### 1.6.5 A hidden database error when adding a condition, allergy or medication (Patient)

On the Medical Profile page, adding a condition, an allergy or a medication gave only a generic "An error occurred" page. The real error was hidden, so we could not tell what was wrong.

We changed the three add actions to catch the exception, write it to the log and show the real database message on the page while we was testing. The message pointed to a column that did not allow NULL: the Category table had a DomainTypeId column that was NOT NULL, while the application only wrote the text column DomainType. We wrote SQL script 015 to make the column nullable and to add unique indexes on category names, and we changed the pages to show a plain message to the user instead of database text once the cause was fixed.

### 1.6.6 Doctors were missing from the patient's Consent page (Patient)

A patient had many pending tests but the Consent page listed no doctors, although doctors had requested those tests. A patient could not see who was testing them or revoke access.

The consent rows were only created in some code paths, so patients whose requests came from other paths (for example seeded data) had no rows. We wrote a backfill script (026) for the existing data and made the Consent page call SyncTreatingDoctors every time it opens, so every doctor with a test request is added automatically. We had to be careful with two rules: a doctor the patient revoked must not be added again, and the limit of five doctors must still be respected. We solved these by comparing against all existing consent rows, including revoked ones.

### 1.6.7 The postal code file had no provinces, so addresses could not be linked to a province (Patient)

Our lecturer asked for normalised addresses (Province, Town, Suburb with postal code). The GeoNames South Africa file (ZA.txt) contains only a place name, a postal code and a latitude and longitude. It has no province or municipality column, so we could not simply import it into our tables.

We first wrote an importer that attached each place to the nearest of about 150 main towns, each with its province, using the latitude and longitude. It worked, but the results near provincial borders were only approximate and the list contained many places nobody needed. We later replaced the import with a smaller, checked list of provinces, towns and suburbs that is loaded by a SQL script, and we added the "Our suburb is not listed" option so that new places can still be entered correctly. The cascading drop-down boxes then worked on the Register, Edit Profile and Admin Profile pages.

### 1.6.8 The doctor search on our Medical Profile page showed nothing useful (Patient)

When we tested "Assign primary doctor" with a real patient, typing a doctor's name found almost nobody, even though the doctors existed in the database.

We found that the search box was filtering a fixed list of a few doctor names written in the page's JavaScript during early development. We replaced it with a server action, SearchDoctors, that reads the real doctors from the database, and made the box show everybody when it is empty and narrow the list as the patient types. The same fix was needed on the Consent page, which had a copy of the same fixed list.

### 1.6.9 Writing dates to SQL DATE columns (Admin & Patient)

EF Core maps SQL DATE columns to DateOnly, but our view models used DateTime for dates such as date of birth, date diagnosed and date started. The first versions of our code did not compile or saved the wrong value.

We now convert explicitly: DateOnly.FromDateTime(value) when writing and value.ToDateTime(TimeOnly.MinValue) when reading. We also stored all times as UTC and wrote the SastTime helper in 1.3.3, so there is one place that converts to South African time.

### 1.6.10 Ensuring unique sample barcodes and handling malformed inputs during test request creation (Doctor)

Solved the risk of duplicate or malformed sample barcodes by replacing reliance on browser inputs with a strict server-side validation and fallback generator (NextBarcode) that guarantees unique tracking numbers before saving to the database.

### 1.6.11 Handling dynamic patient medical data and multi-doctor consent restrictions in test requests (Doctor)

Overcame the challenge of filtering historical test records across multiple doctors by implementing client-side JSON rendering with automatic conditional consent checks to enforce privacy restrictions and display warning alerts when needed.

### 1.6.12 The patient details tabs did not switch when clicked (Lab Technician)

On the Capture Result and Verify Result pages the patient’s conditions, allergies and medication are shown in tabs above the result form. Clicking a tab highlighted it, but the content underneath never changed, and no error was shown on the page.

I opened the browser console and found an error from Bootstrap’s own JavaScript. I followed it into bootstrap.bundle.js and saw that when a tab is shown, Bootstrap looks for the closest parent element with the class nav or list-group. My tab wrapper only had my own class, pt-tabs, so Bootstrap found nothing and stopped before it reached the code that switches the content. I added the nav class to the wrapper of every tab group, and the tabs have switched correctly since. I learned to read the framework’s source when a component fails silently instead of only changing my own code.

### 1.6.13 A test request was only marked Completed after every result was verified (Lab Technician)

When I checked the Lab Technician pages against the specification, I noticed that a test request stayed In Progress until every one of its results had been verified by a second technician. The doctor therefore saw the request as still in progress long after all the tests had been performed.

I read the request statuses in the specification again. A request is Completed once the results of all its tests have been captured; verification is a separate step that is tracked per test and that triggers the e-mail to the doctor. I moved the rule into CaptureResult: when no test on the request is still awaiting a sample, collected or being tested, the request becomes Completed. A rejected sample does not block this, because that test will never be captured. The e-mail with the PDF is still only sent when every test is verified, as shown in 1.3.9.

### 1.6.14 Pop-up windows were not centred on the page (Lab Technician)

All the pop-up windows (modals) in the Lab Technician portal, for example the "View all" lists on the dashboard and the review history on the Verification History page, opened noticeably to the left of the page content, and the large ones were partly covered by the sidebar.

Bootstrap centres a modal against the whole browser window, but in my layout the dark sidebar on the left takes up part of the window, so the visible page is only the area to the right of it. I added one CSS rule to the Lab Technician layout that gives every modal a left padding equal to the width of the sidebar. Bootstrap then centres the window inside the remaining space, and because the rule is on the base modal class it works for every modal size without changing each page.

### 1.6.15 Table headings were missing on the second page of the PDF reports (Lab Technician)

When I generated a technician report for a longer date range, the report ran over more than one page, and the second page started straight with rows of results without any column headings, so it was hard to tell which column was the value and which was the normal range.

The tables in PdfService were built by hand as one row after another, and the heading was only another row at the top, so QuestPDF had no way of knowing that it was a heading. I rewrote the report tables to use the QuestPDF Table element with column definitions and a separate table.Header section, which QuestPDF repeats on every page. While doing this I also aligned numbers and dates to the right and status columns to the centre, and moved a result’s note into a cell that spans the full width of the table so it stays inside the table border. The colours and branding of the reports did not change.

### 1.6.16 Moving the Lab Manager pages from in-memory data to the shared database (Lab Manager)

I first built all the Lab Manager pages against temporary lists in memory, so that I could work before the group database was ready. When we moved to one shared SQL Server database, the Entity Framework classes generated from the database did not match my pages: the names were different, some fields I needed did not exist (for example an order number, a supplier contact person and the reason and date when an order was cancelled), and the database used different status values, such as Placed and PartiallyReceived instead of Ordered and Partial.

The specification does not allow the database to be generated by Entity Framework, so I could not use migrations. I wrote numbered SQL scripts that only add the Lab Manager columns and constraints, so that they could be run safely without touching my teammates' tables. I kept my view models, so the pages did not have to be rewritten, and mapped them to the database classes in the controllers, with small functions that translate the status codes in both directions. I then moved one controller at a time and tested each page before moving on.

### 1.6.17 Edit pages that did nothing when Save was clicked (Lab Manager)

After the move to the database, the Edit Consumable page loaded correctly, but clicking Save did nothing: there was no error message, no redirect and no notification. The Edit Supplier page had a similar problem. Because no message was shown, I first thought the form was not being posted at all.

When I looked at the ModelState errors I found the real cause. The project uses nullable reference types, so a navigation property such as Consumable.Supplier, which the form never posts, was treated as required and made the model invalid. On the Supplier page the name field was disabled, and disabled fields are not posted, so the name was also reported as missing. I excluded these fields from validation and used the stored values instead. I also learned that Hot Reload had not applied some of my changes, so after this I always restarted the application before testing a controller change.

### 1.6.18 The preloaded data was changed by people using my login (Lab Manager)

After I loaded the haematology data that the preload requirements ask for, the Lab Manager dashboard suddenly showed 3 test categories instead of 6 and 54 test types instead of 34. Old non-haematology tests were active again and some technicians were archived. Because the database is shared, I first suspected a teammate's script.

I wrote a read-only SQL script to compare the data with what I had loaded and to list the activity log since then. The log showed that every change had been made on the Lab Manager pages using my own account: teammates had used my login details to test the system on the published site. I put the data back through the pages, changed my password, and asked the group to test with the other Lab Manager accounts. I also made my preload script safe to run again, so that the data can be restored quickly before judging.

### 1.6.19 Merge conflicts and locked files when using Git in OneDrive (Lab Manager)

Our project folder is inside OneDrive. When a teammate renamed the database context and the model folder while I was changing my controllers, my push was rejected, the pull ended in merge conflicts, and git merge --abort failed because OneDrive and Visual Studio were holding some files open. For a while my commit with the login protection was on my computer but not on the server.

I paused OneDrive syncing and closed Visual Studio, reset my folder to the last good commit, pulled the latest version and applied my Lab Manager files again on top of it. I then checked with git status that only my files had changed and confirmed with git log origin/main that my commit had reached the server. Since then I always pull before I start working and check git status before every commit.